SOFTWARE TESTING PRACTICAL REPORT

TUTORIAL – 14
Selenium Web Automation

Browser Navigation • Login Form Handling • Product Interaction • Cart Validation

Student Name	Harshita Malviya
Student ID	2504030100426
Practical	Tutorial 14 — Selenium Web Automation
Application Under Test	SauceDemo
Programming Language	Python
Tools	PyCharm, Selenium WebDriver, Google Chrome
Test Script	tutorial14.py

Prepared using the supplied Tutorial 14 lab manual, Selenium script, and screenshots.

1. Aim

To create Selenium automation scripts in Python for browser navigation, form handling, web element interaction, and validation testing.

2. Practical Description

This practical automates the SauceDemo workflow using PyCharm, Python, Selenium WebDriver, and Google Chrome. The script opens the website, enters valid login credentials, validates the inventory URL, adds the Sauce Labs Backpack to the cart, opens the cart, validates the cart URL, and closes the browser.

3. Test Environment

Parameter	Details
Project	Selenium Web Automation
Application	SauceDemo — https://www.saucedemo.com/
Module	Login and Product Navigation
SDLC Model	Agile (as specified in the lab manual)
Language / IDE	Python / PyCharm
Automation Tool	Selenium WebDriver
Browser	Google Chrome
Wait Strategy	WebDriverWait with a 10-second timeout
Test Credentials	standard_user / secret_sauce
Product	Sauce Labs Backpack

4. Test Scenario and Expected Results

ID	Test Scenario	Expected Result
TC-14-01	Browser navigation	SauceDemo login page opens
TC-14-02	Username entry	Username is entered into the username field
TC-14-03	Password entry	Password is entered into the password field
TC-14-04	Login validation	User is redirected to the inventory page
TC-14-05	Product interaction	Sauce Labs Backpack is added to cart
TC-14-06	Cart navigation	Cart page opens and URL contains 'cart'

5. Automation Workflow

• Initialize Chrome WebDriver and maximize the browser window.

• Open https://www.saucedemo.com/ and print a website-opened message.

• Wait until the username field is visible and enter standard_user.

• Enter secret_sauce and click the login button.

• Wait for the URL to contain inventory and assert the inventory URL.

• Wait for the backpack Add to Cart button to be clickable and select it.

• Open the shopping cart and assert that the current URL contains cart.

• Close the browser in the finally block.

6. Web Element Identification

Element	Locator Type	Locator Value	Purpose
Username	ID	user-name	Enter username

Element	Locator Type	Locator Value	Purpose
Password	ID	password	Enter password
Login button	ID	login-button	Submit login form
Backpack button	ID	add-to-cart-sauce-labs-backpack	Add product
Shopping cart	Class Name	shopping_cart_link	Open cart

7. Source Code Reviewed

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
driver.maximize_window()
wait = WebDriverWait(driver, 10)

try:
driver.get("https://www.saucedemo.com/")
print("Website opened")

username = wait.until(
EC.visibility_of_element_located((By.ID, "user-name"))
)
username.send_keys("standard_user")
driver.find_element(By.ID, "password").send_keys("secret_sauce")
driver.find_element(By.ID, "login-button").click()

wait.until(EC.url_contains("inventory"))
assert "inventory" in driver.current_url
print("Login Validation: PASS")

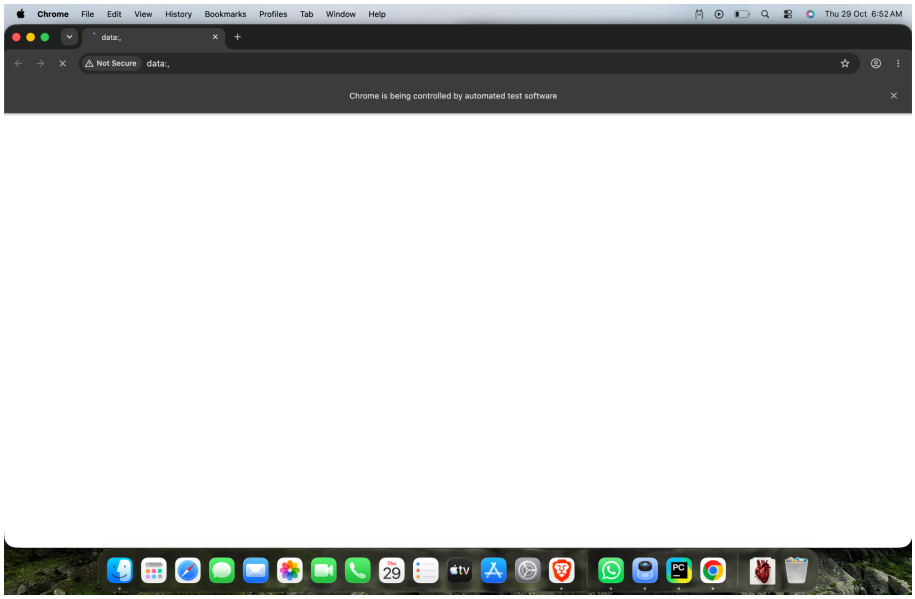
wait.until(EC.element_to_be_clickable(
(By.ID, "add-to-cart-sauce-labs-backpack")
)).click()
print("Product Selection: PASS")

driver.find_element(By.CLASS_NAME, "shopping_cart_link").click()
assert "cart" in driver.current_url
print("Cart Validation: PASS")
finally:
driver.quit()

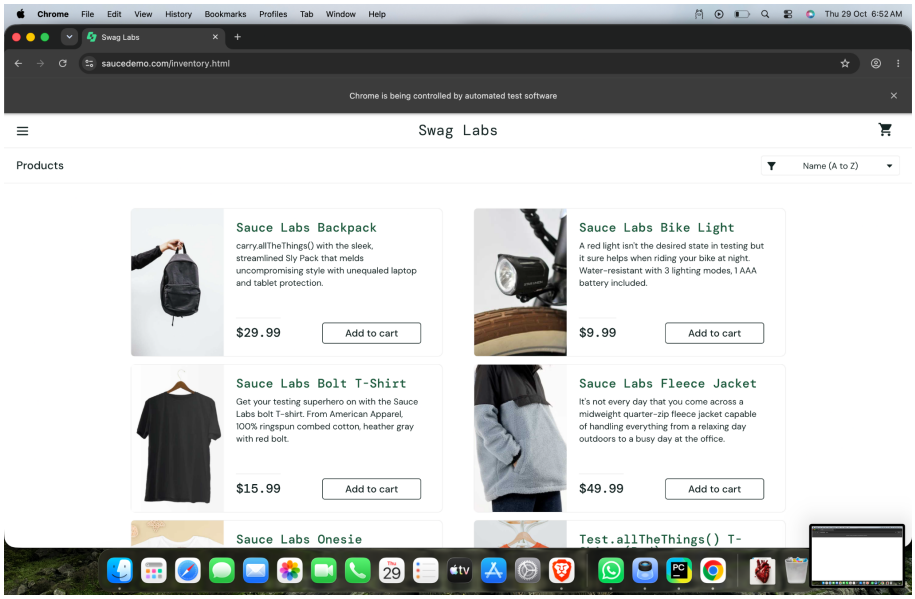
print("Tutorial 14 Completed Successfully")
```

8. Screenshot Evidence

Screenshot 1 — Browser loading state



Screenshot 2 — SauceDemo inventory page



Screenshot 3 — Console output supplied

```
AI-Assisted Login Test: PASS
Requirement Validation: PASS

Process finished with exit code 0
```

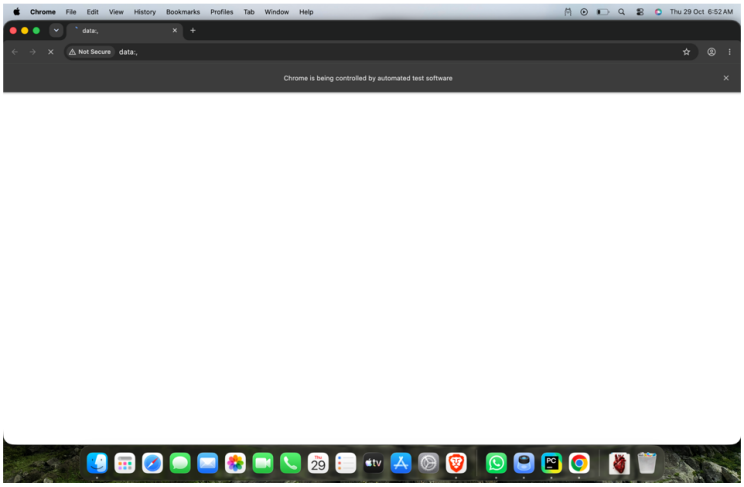
9. Screenshot Collage

The collage combines the three supplied images into one evidence sheet. The console image shows PASS messages for an AI-assisted login test and requirement validation; it does not show the exact Tutorial 14 print messages, so it is treated as related execution evidence rather than proof of every step in this script.

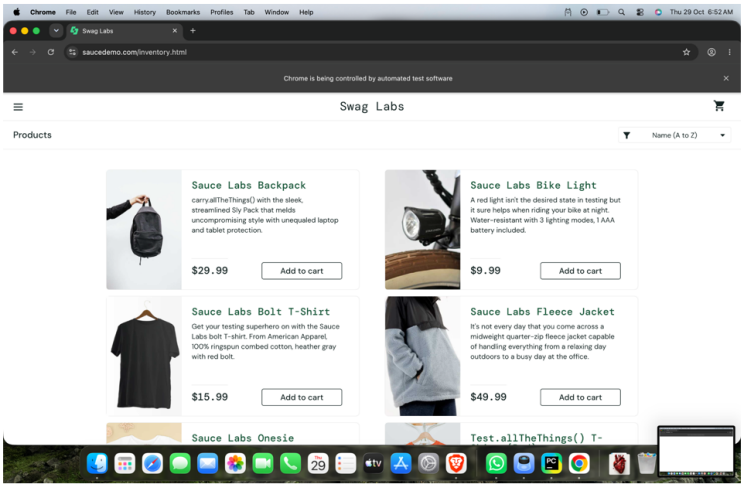
TUTORIAL 14 | SELENIUM WEB AUTOMATION

SauceDemo browser evidence and automation console

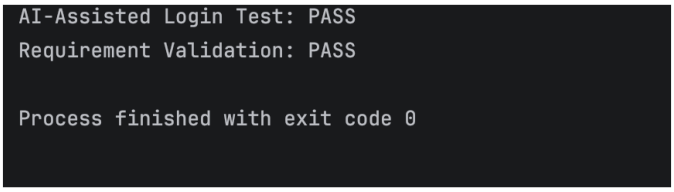
Screenshot 1 ⊟ Browser loading / data page



Screenshot 2 ⊟ SauceDemo inventory / products page



Screenshot 3 ⊟ Console PASS messages



10. Execution Summary

Check	Evidence / Result
Browser launched	Browser screenshots supplied
Products page visible	SauceDemo inventory page visible
Login validation	Console shows a related login PASS message
Product selection	Not directly confirmed by the supplied screenshots
Cart navigation	Not directly confirmed by the supplied screenshots
Exact Tutorial 14 console output	Not supplied

11. Observations

• The supplied products screenshot shows the SauceDemo inventory page.

• The supplied browser loading screenshot shows a blank data: page and does not verify a successful SauceDemo navigation by itself.

• The console screenshot shows “AI-Assisted Login Test: PASS”, “Requirement Validation: PASS”, and process exit code 0.

• The supplied Tutorial 14 script contains explicit URL assertions for the inventory and cart pages and closes the browser in a finally block.

12. Result and Conclusion

The submitted Tutorial 14 script is designed to automate the main SauceDemo shopping flow: login, inventory validation, backpack selection, and cart navigation. The screenshots provide evidence of the products page and a related successful login console run. Because the exact console output for this script and a cart-page screenshot were not supplied, those steps should be captured before marking every workflow step as independently verified.

13. Professional QA Recommendations

• Capture screenshots at the login page, inventory page after adding the backpack, cart page, and PyCharm console.

• Add a visible-element assertion for the Products heading in addition to URL validation.

• Verify the backpack is present in the cart, not only that the URL contains cart.

• Keep the finally block so Chrome is closed even when a test fails.

14. Questionnaire

Q1. What is Selenium? Selenium is an open-source framework used to automate web browsers and test web applications.

Q2. What is Selenium WebDriver? An API used to control and interact with web browsers programmatically.

Q3. What is ChromeDriver? A driver component that allows Selenium to communicate with Google Chrome.

Q4. Which method enters text into a textbox? send_keys()

Q5. Which method clicks a button? click()